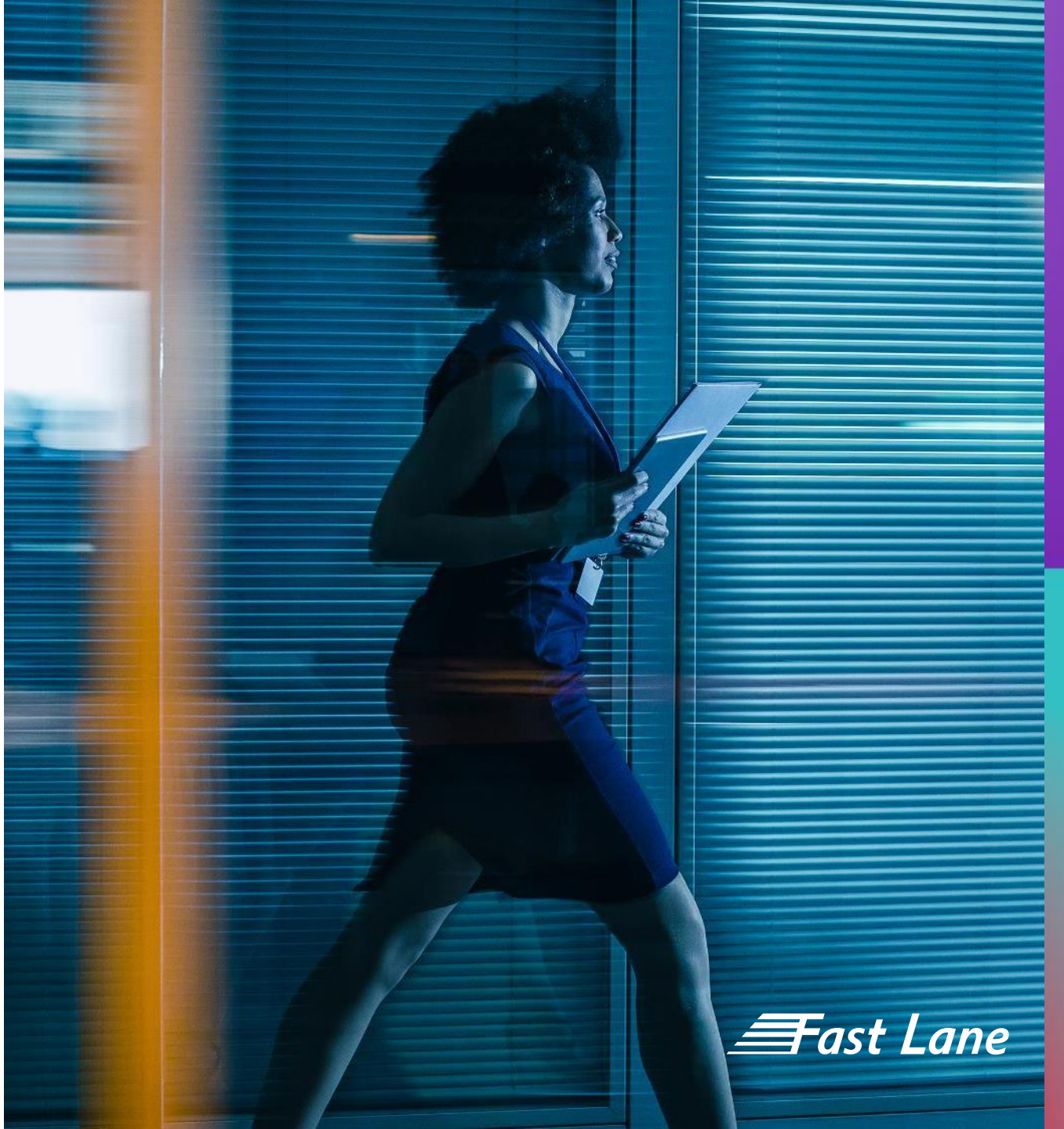
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 4

Consuming & Designing MCP Tools

Servers, transports, apps and tools Claude will pick



01

Intro

Architecture, transports and the servers you use

02

MCP Server

A C# server over Streamable HTTP

03

MCP Apps

Images and interfaces from a tool

04

Artifact Connectors

Live data in a published page

05

Tool Design

Tools Claude will actually pick

Consuming and designing MCP tools

Servers, transports, apps and tools Claude will pick

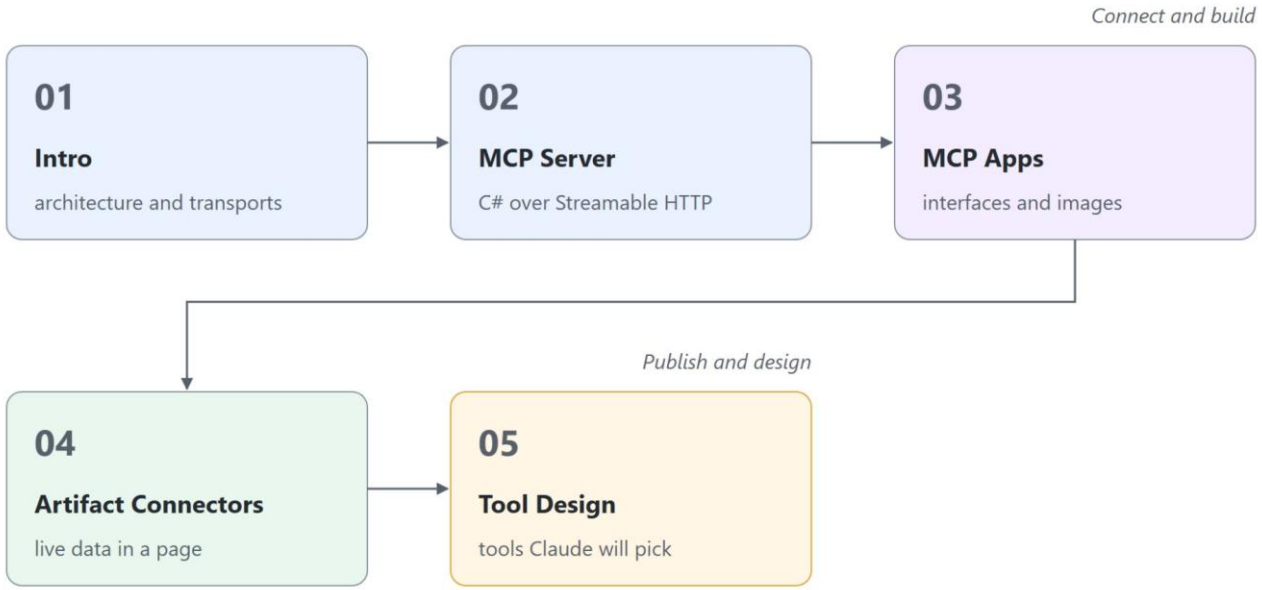
- **Consume**: connect hosted and local MCP servers through one `.mcp.json` entry
- **Build**: a C# MCP server over Streamable HTTP, and tools that return images
- **Design**: names, descriptions and error contracts that make Claude pick the tool



Overview

The path through MCP

- Start with the host, client and server model and the two transports
- Run a **C# MCP server** and verify its tools with the MCP Inspector
- Return pictures from a tool and render interfaces with **MCP Apps**
- Call **connectors** from a published artifact page at view time
- Finish with **tool design**: granularity, descriptions and error contracts



01

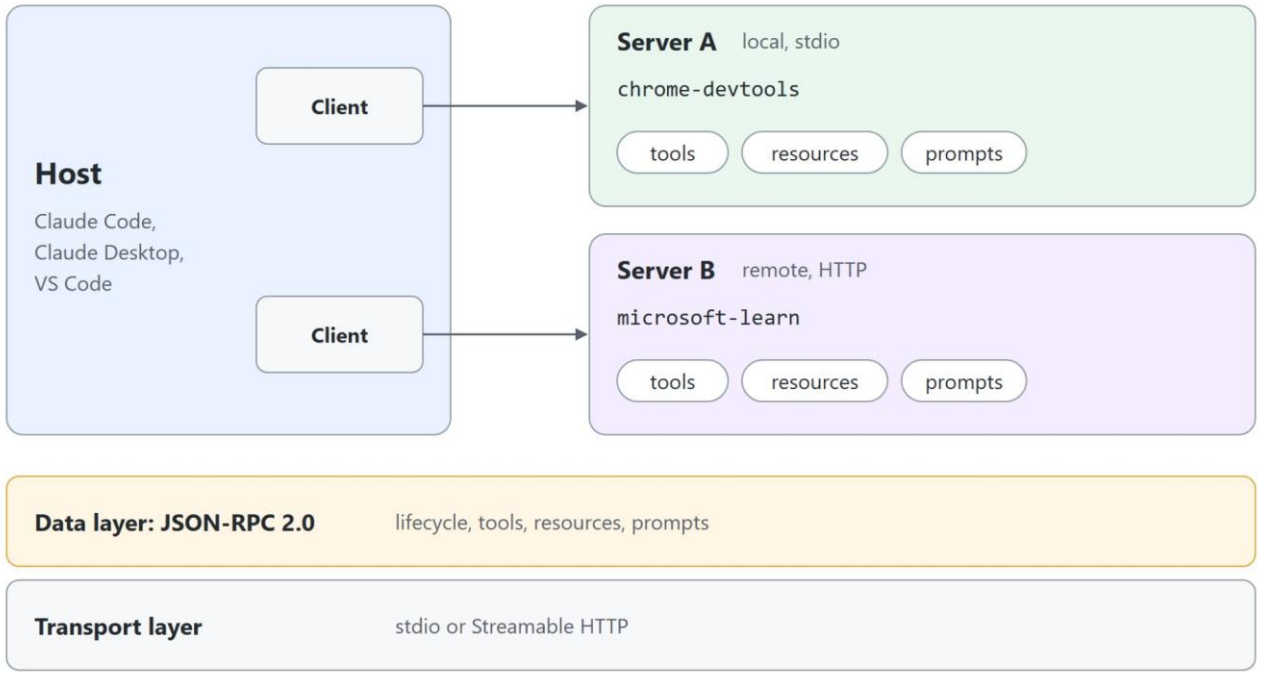
Intro

Architecture, transports and the servers you use

MCP architecture

Host, client and server, and the two protocol layers

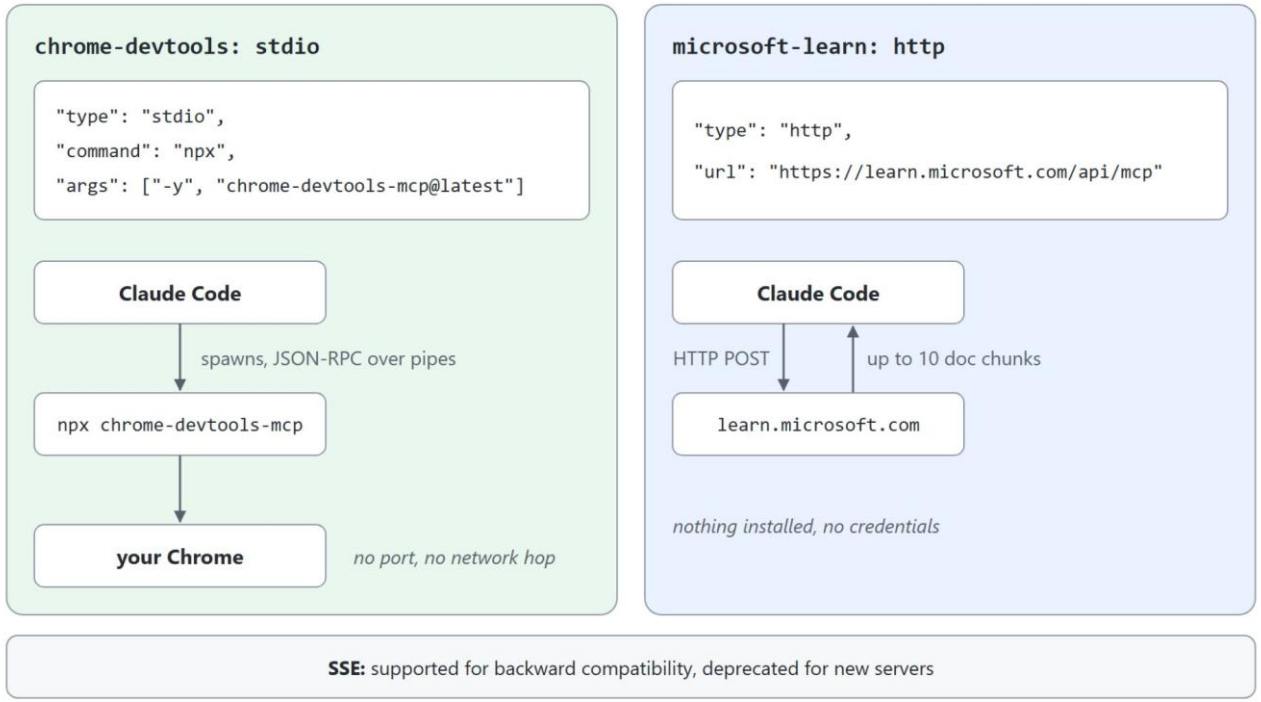
- The **host** owns the session and creates one **client** per server
- A **server** exposes tools, resources and prompts and runs on its own
- The data layer is **JSON-RPC 2.0**, the same over every transport
- Claude Code names MCP tools `mcp__<server>__<tool>`
- Servers can ask back: **sampling** for a completion, **elicitation** for user input



stdio and Streamable HTTP

Two entries from this repo's .mcp.json

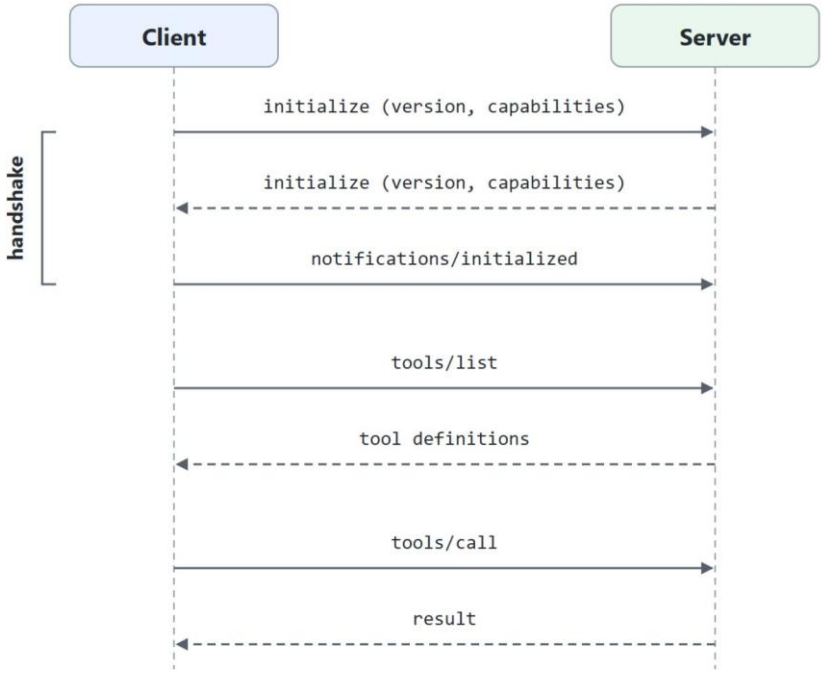
- **stdio**: Claude Code spawns the server and talks JSON-RPC over pipes
- **Streamable HTTP**: POST requests with optional streaming, required for remote servers
- HTTP carries bearer tokens, API keys and **OAuth 2.0**
- `microsoft-learn` answers from current docs; `chrome-devtools` drives a real browser
- `/mcp` shows status, tool counts and the sign-in actions per server



Session lifecycle and the MCP Inspector

Handshake first, then discovery and calls

- Every session opens with **initialize**: protocol version and capabilities
- Incompatible versions end the connection before any other message
- `tools/list` discovers, `tools/call` invokes
- A server that declares `listChanged` can announce new tools mid-session
- The **MCP Inspector** exercises a server outside any host, showing raw JSON-RPC



MCP Inspector

```
npx
@modelcontextprotocol/
inspector --cli
```

browser UI | CLI | TUI

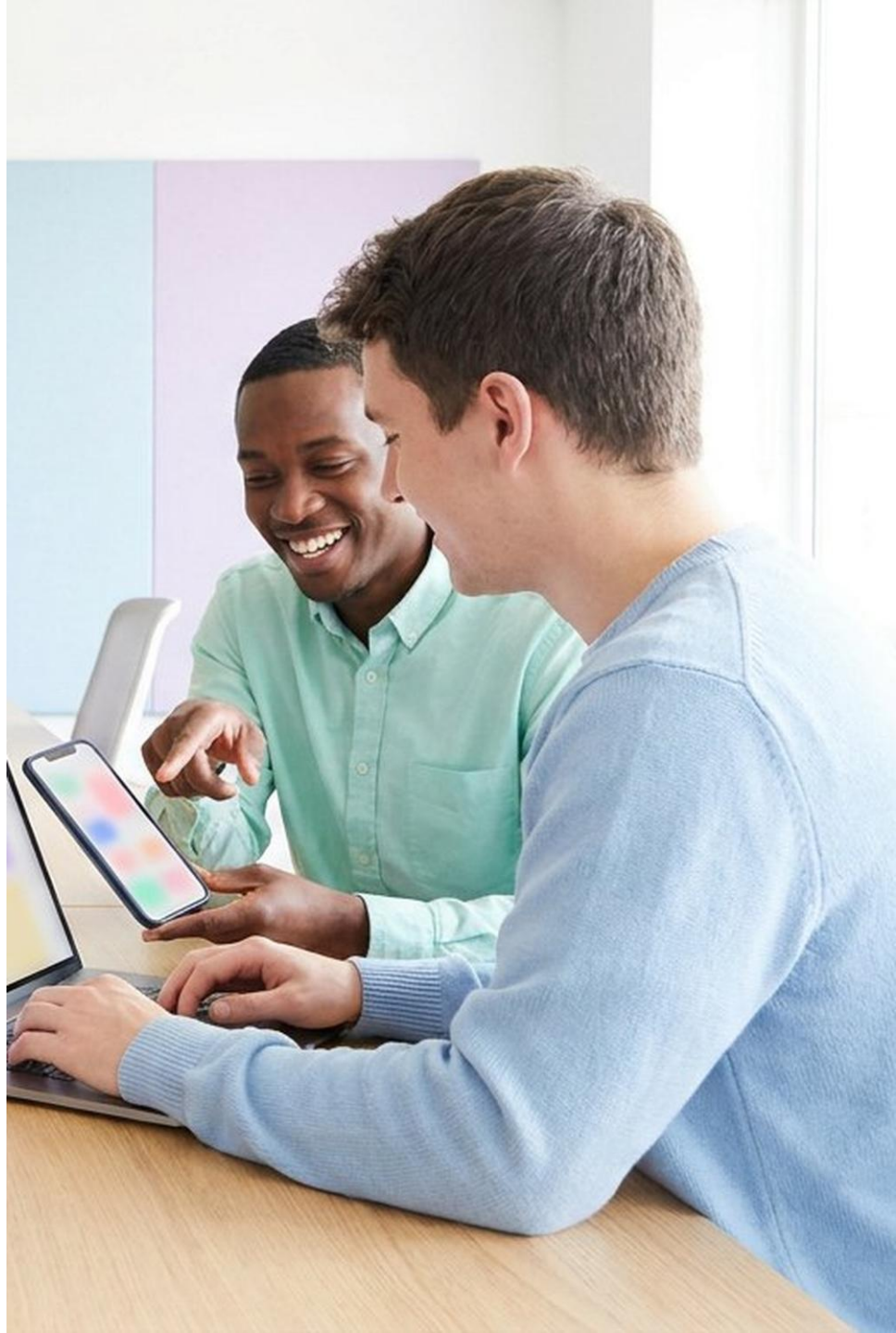
`notifications/tools/list_changed`

requires `listChanged: true`

MCP servers for developers

Browser, cloud, repository and documentation tools

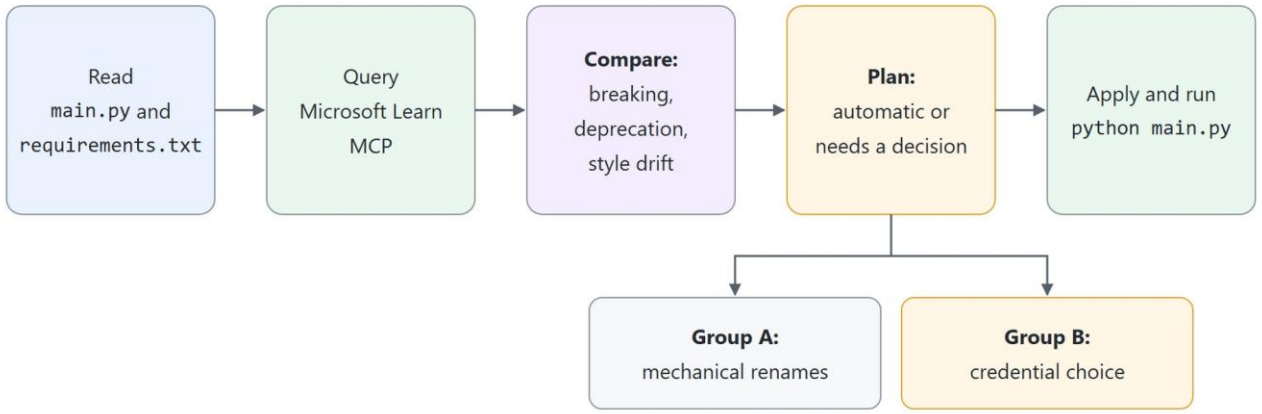
- **Playwright**: describe the browser test, Claude writes and runs it
- **Azure Deploy**: requirements become Bicep or Terraform with cost estimates
- **GitHub**: list reviews, read diffs and comment on specific lines
- **Microsoft Learn**: answers grounded in the current official docs
- Find more in the **GitHub MCP Registry**, **Smithery** and the official registry



Upgrading a framework with Microsoft Learn MCP

Current documentation for a fast-moving framework

- The Microsoft Agent Framework is **pre-release**: breaking changes are frequent
- Claude compares the starter against the **live docs** and classifies every gap
- Mechanical renames apply automatically; **credential choices** wait for you
- Done when `python main.py` answers from the Foundry project



`demos/04-mcp/01-intro/maf-migration`

02

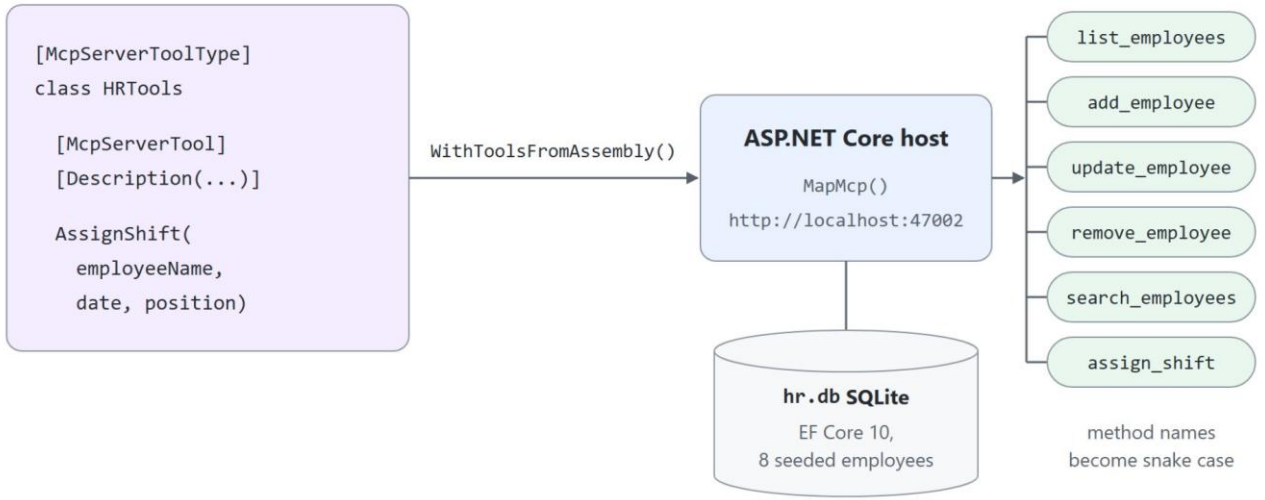
MCP Server

A C# server over Streamable HTTP

An HR MCP server in C#

Six attributed methods become six tools

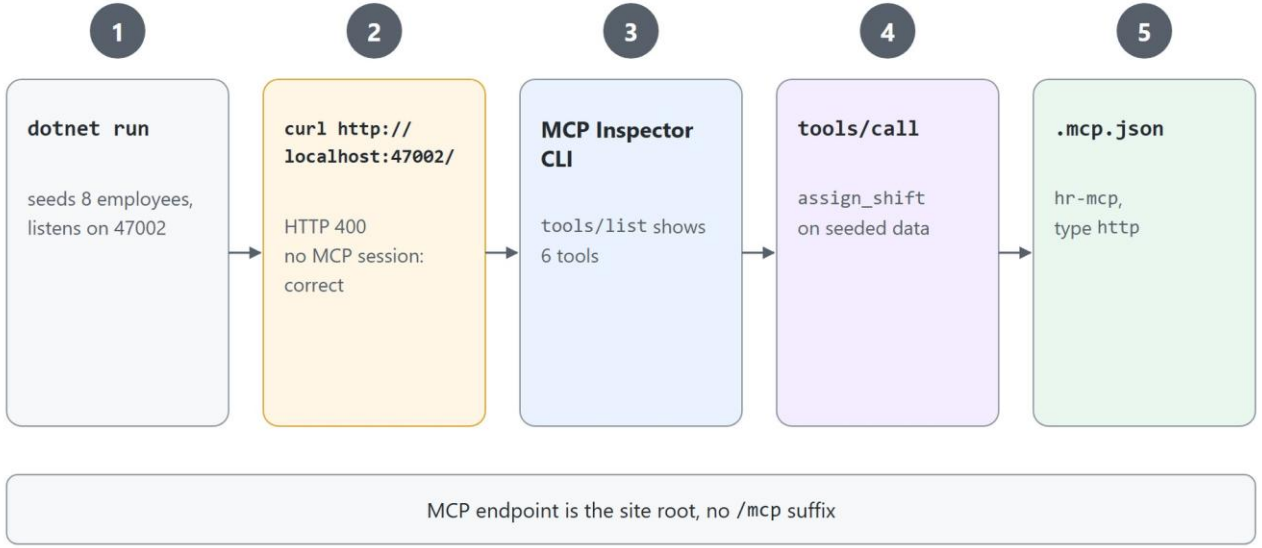
- C# .NET 10 and ASP.NET Core with `ModelContextProtocol` **1.2.0**
- **Streamable HTTP** on port 47002, EF Core SQLite for employees and shifts
- `[McpServerTool]` and `[Description]` text is what the model reads
- `WithToolsFromAssembly()` registers all six tools, `MapMcp()` exposes them
- Method names arrive in snake case: `AssignShift` becomes **`assign_shift`**



Verifying and registering the server

From dotnet run to a .mcp.json entry

- `dotnet run` seeds eight employees and binds **port 47002**
- A plain `curl` gets **HTTP 400**: the right answer to a non-MCP request
- The **MCP Inspector CLI** lists the six tools and calls one against real data
- The endpoint is the site root, with no `/mcp` suffix
- One `.mcp.json` entry hands the tools to Claude Code, no client code



03

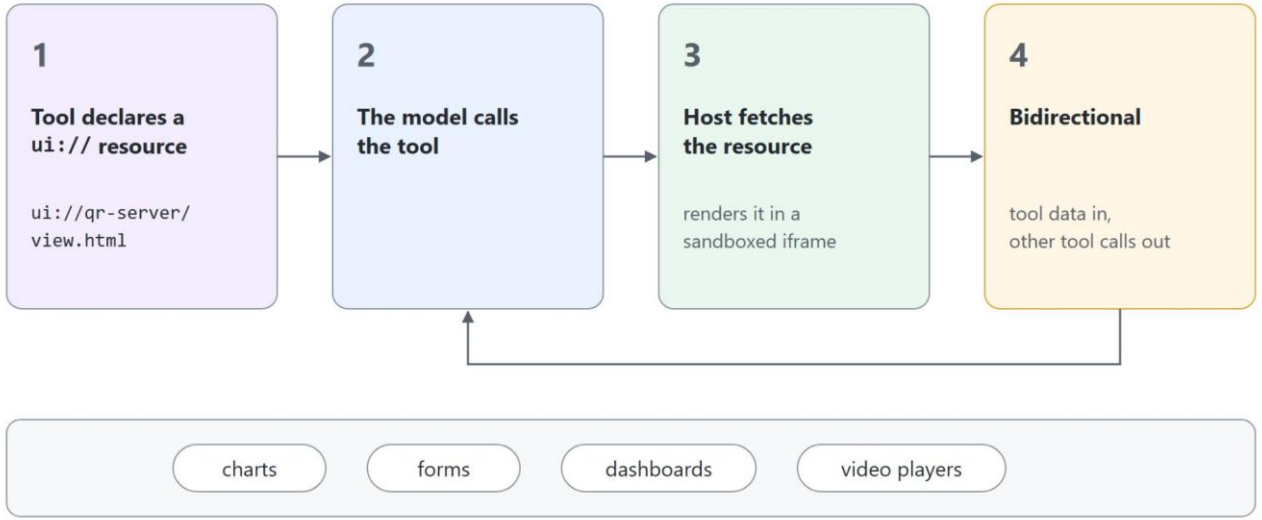
MCP Apps

Images and interfaces from a tool

MCP Apps

Interactive interfaces rendered inside the chat

- MCP gives the tools and data; **MCP Apps** give the visual interface
- A tool declares a `ui://` resource that holds its HTML
- The host renders it in a **sandboxed iframe** in the conversation
- The view receives tool data and can call other tools through the host
- Runs in Claude, ChatGPT, VS Code and other compliant hosts

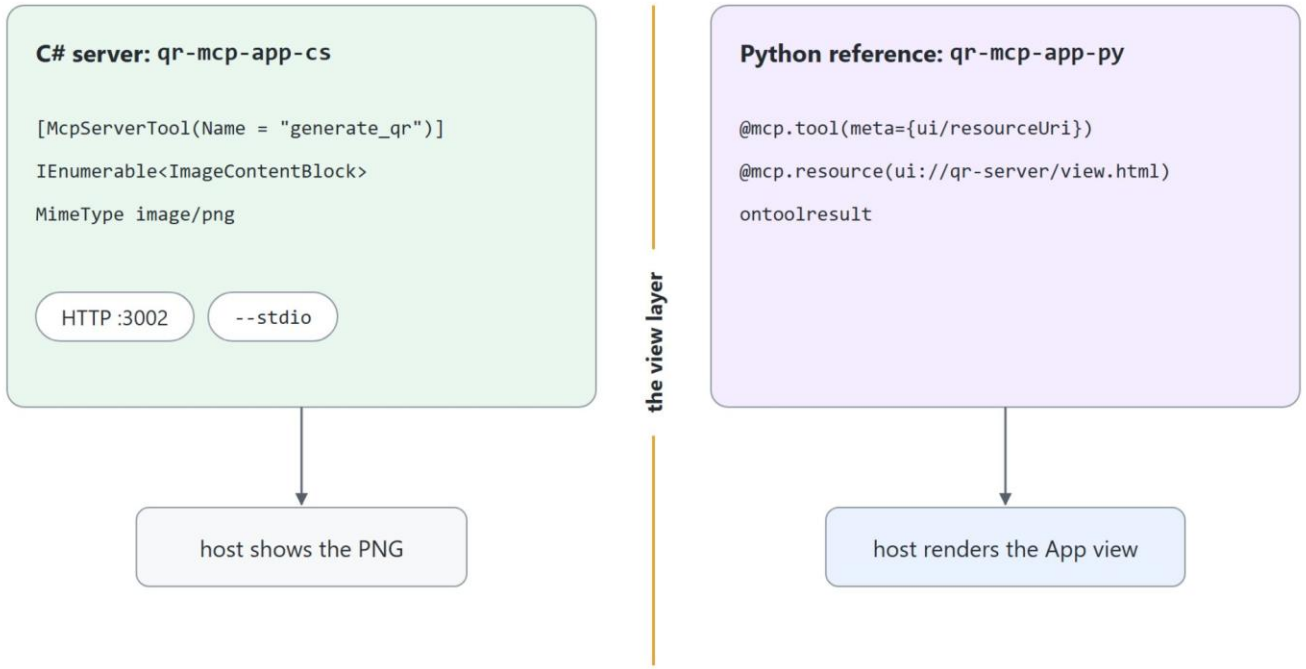


spec stable as of 2026-01-26

A tool that returns an image

The C# QR code server and the missing view layer

- `generate_qr` returns an **`ImageContentBlock`**, a base64 PNG
- Streamable HTTP on **port 3002**, or stdio with `--stdio`
- Box size, border, error correction and colours are optional arguments
- The C# server registers a tool only: hosts show the picture, no App surface
- The Python reference adds the **`ui://`** view that makes it an MCP App



04

Artifact Connectors

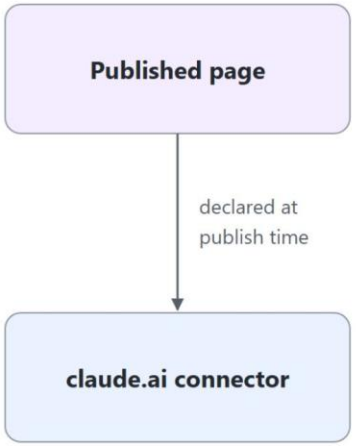
Live data in a published page

Live data in a published page

Which MCP servers a page can call

- Connector calls are the one exception to an artifact having **no backend**
- Local `.mcp.json` servers feed the build but cannot be called once published
- Only **claude.ai connectors** can be called when someone opens the page
- Claude declares the allowed connectors **at publish time**
- Needs Claude Code **v2.1.209** or later on Pro, Max, Team or Enterprise

Source	Feeds the build	Called at view time
Local stdio server in `.mcp.json`	✓	✗
Remote HTTP server registered locally	✓	✗
Connector on your claude.ai account	✓	✓

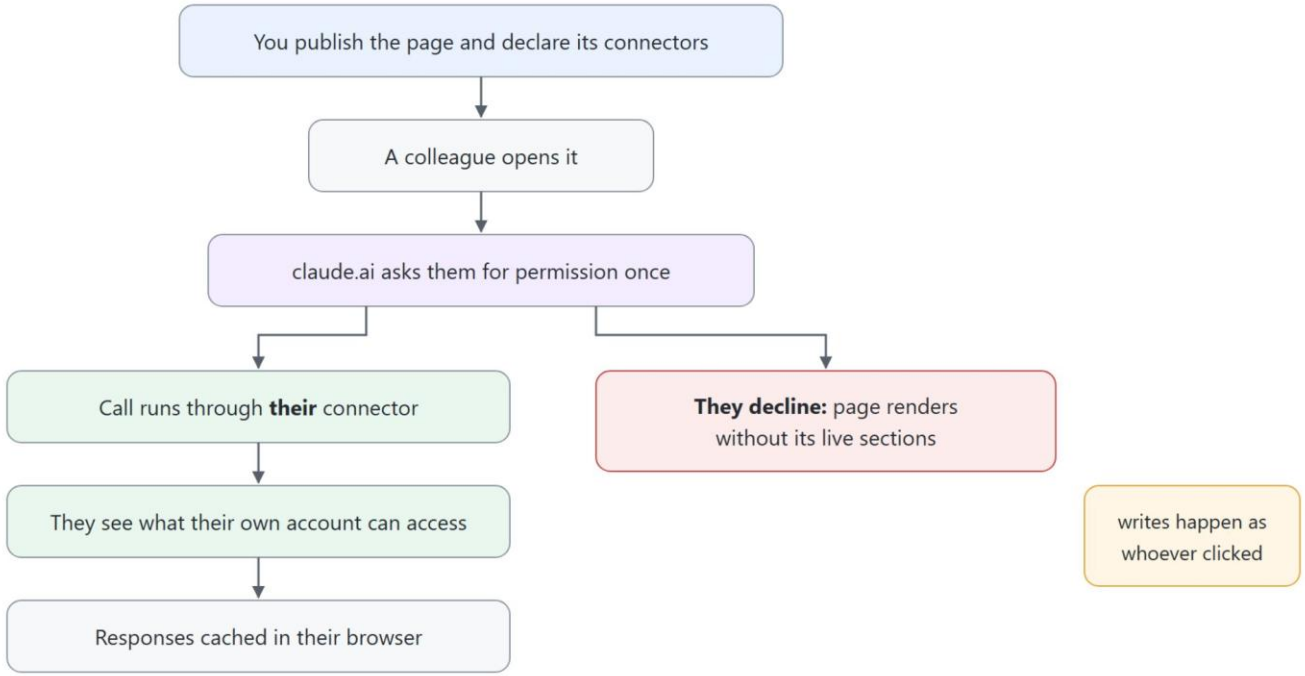


the declaration is the security boundary

Calls run as the viewer

Why two people can see different rows

- Each call goes through the **account of whoever opens the page**
- claude.ai asks the viewer for permission once per page load
- The page never sees anyone's credentials
- A control that writes, writes as **the person who clicked it**
- Responses are **cached per viewer**: first paint from cache, then fresh data



Sharing a connector-backed page

No public link, and a fallback for every live section

- **Never a public link**, on any plan: there is no signed-in viewer to run calls as
- Pro and Max keep it private; Team and Enterprise share **inside the organization**
- Give each live section a **fallback** that names the connector it needs
- One viewer sees data, another nothing: the second has not added the connector
- Nobody sees data: an Owner turned off the artifact connectors toggle



05

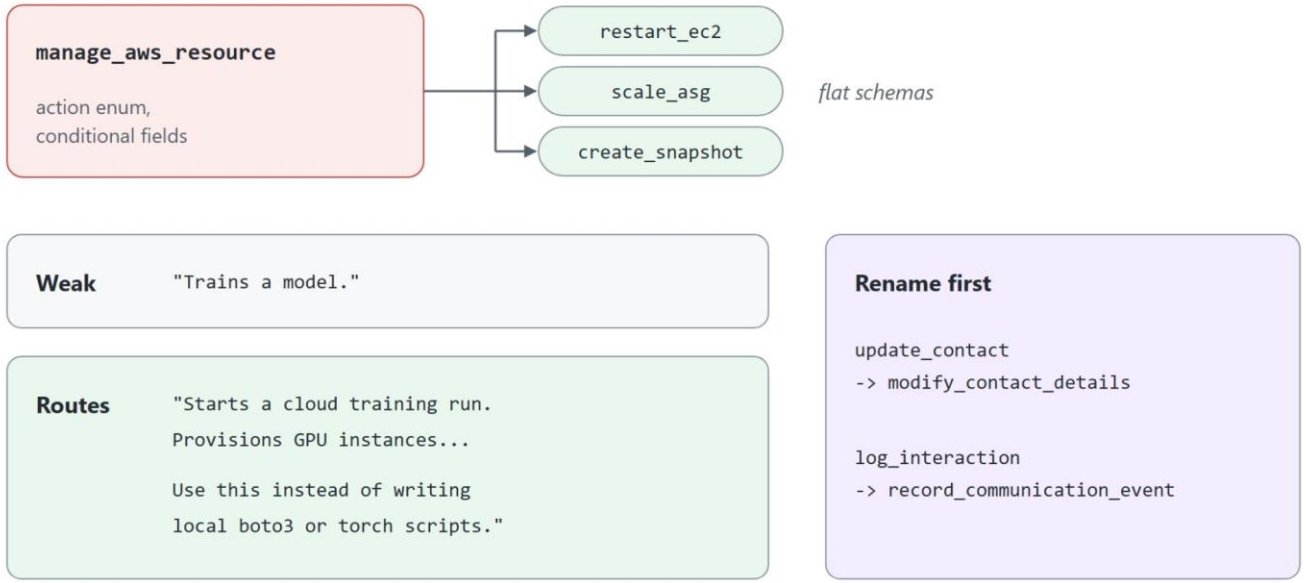
Tool Design

Tools Claude will actually pick

Granular tools and routing descriptions

Names, descriptions and schemas Claude reads

- Split one generic tool into **purpose-built tools** with flat schemas
- Split by **output contract** when the right call yields the wrong answer
- The **description** says what the tool handles and why it beats a script
- Rename near-synonyms first: they get conflated whatever the descriptions say
- Expose bulk reads as a **resource**, keep tools for actions and lookups



How a tool should fail

The error result is an instruction to the model

- `isError`, `errorCategory` and `isRetryable` decide retry, escalate or stop
- **permission** and **validation** failures are never retryable
- **transient** failures, such as timeouts and rate limits, are
- A retryable permission denial loops against a wall
- A search with no matches ran correctly: **success with zero rows**

```
{
  "isError": true,
  "errorCategory": "permission",
  "isRetryable": false,
  "message": "Insufficient
    privileges... Escalation
    to Tier 3 required."
}
```

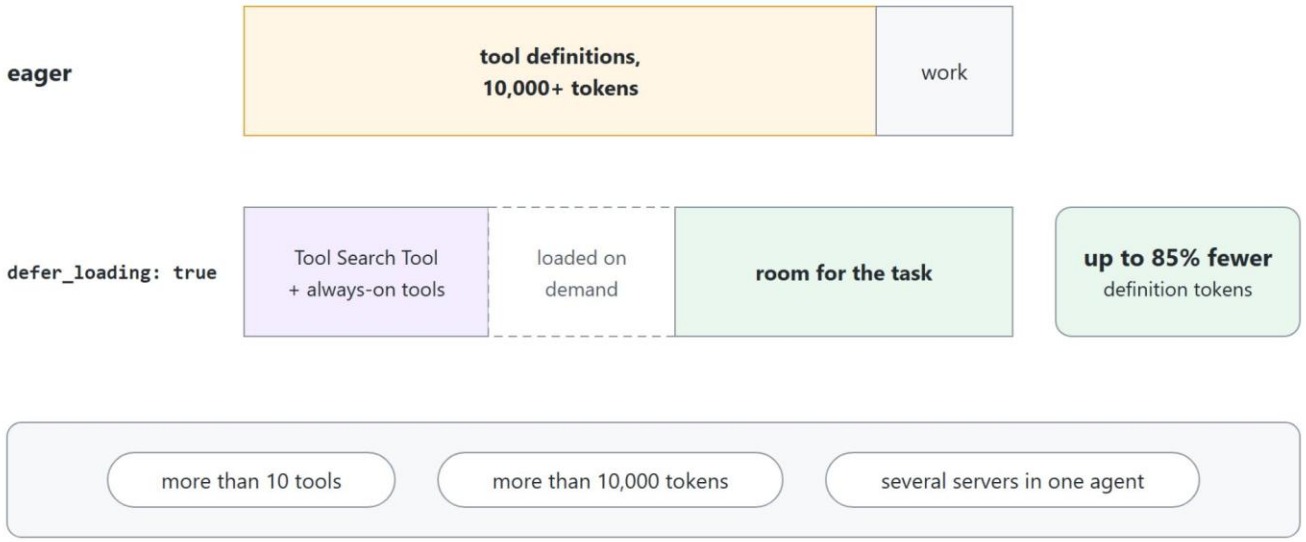
Failure	Category	Retryable
Caller lacks the privilege	`permission`	**false**
Malformed input, wrong date format	`validation`	**false**
Upstream timeout, rate limit	`transient`	**true**

zero matches is a success with zero rows

Deferred tool loading

When the tool count outgrows the context

- Tool definitions fill the context window **before any work starts**
- Mark most tools `defer_loading: true`
- The agent starts with the **Tool Search Tool** plus the always-on tools
- Definitions load on demand: up to **85%** fewer tool-definition tokens
- Use it past **10 tools**, 10,000 tokens of definitions, or several servers



Takeaways

What to carry into the CLI module

- **stdio** for local processes, **Streamable HTTP** for anything remote or shared
- Verify a server with the **MCP Inspector** before Claude Code sees it
- Return images or a `ui://` view when text is the wrong result
- Published pages call **claude.ai connectors** only, and run as the viewer
- Design the **name, description and error contract**; defer loading past ten tools

